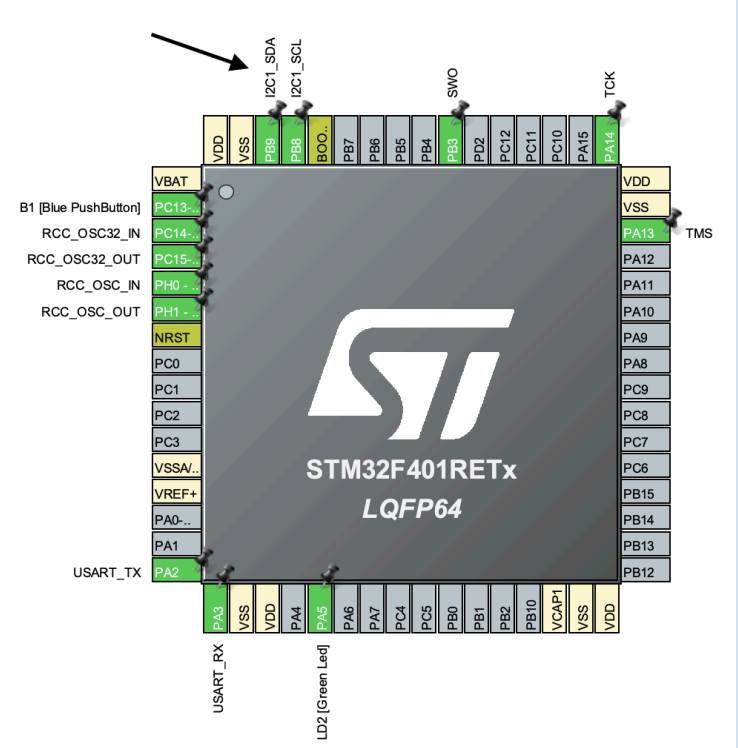
Mark	7/11

Team name:	B9		
Homework number:	Homework 07		
Due date:	04/11/2025		
Contribution:	No	Partial	Full
Pica Mirko			✅
Prendin Christian			✅
Pignataro Gabriele			✅
Moazenkakhki Mostafa			✅
Yarrabbotula Sunandha			✅
Notes:			

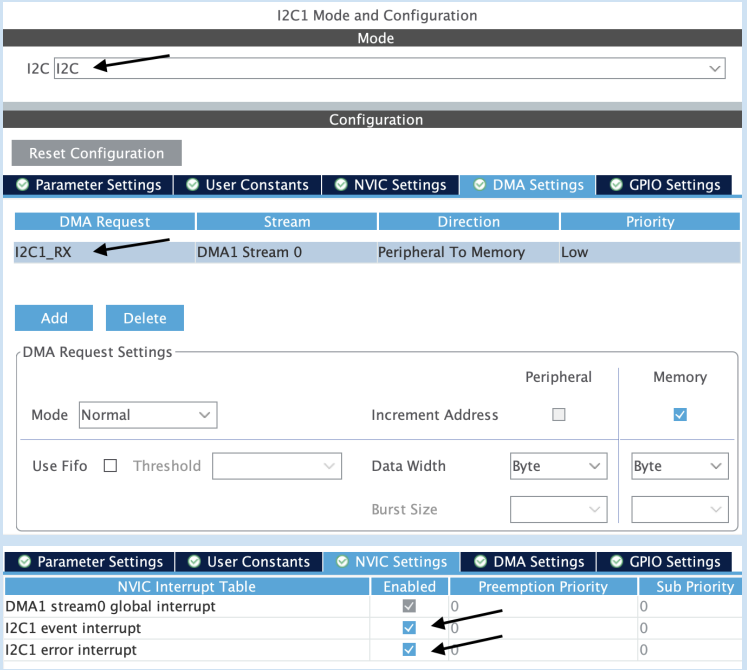
Project name	I2C Thermometer		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			✅

Part 1. Complete the I2C thermometer project 1b.
Verify that the binary to Celsius conversion procedure you come up with works for both positive and negative numbers (see datasheet table 10).

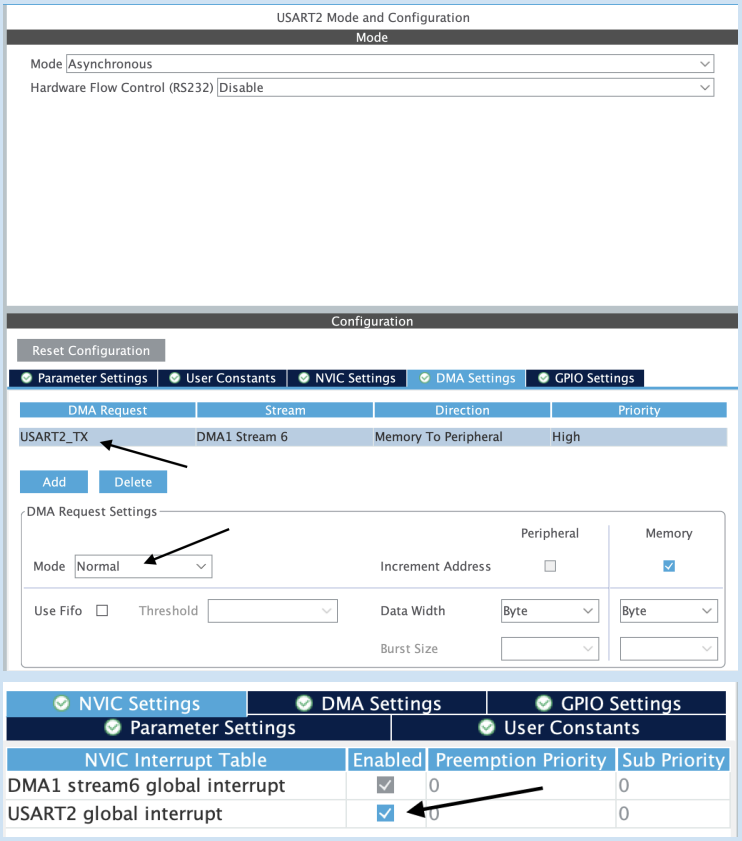
From the “Hands-on Lab Schematics.pdf” we found out that SDA is connected to pin PB9 and SCL is connected to pin PB8. So first we set them up in the ioc configurator.



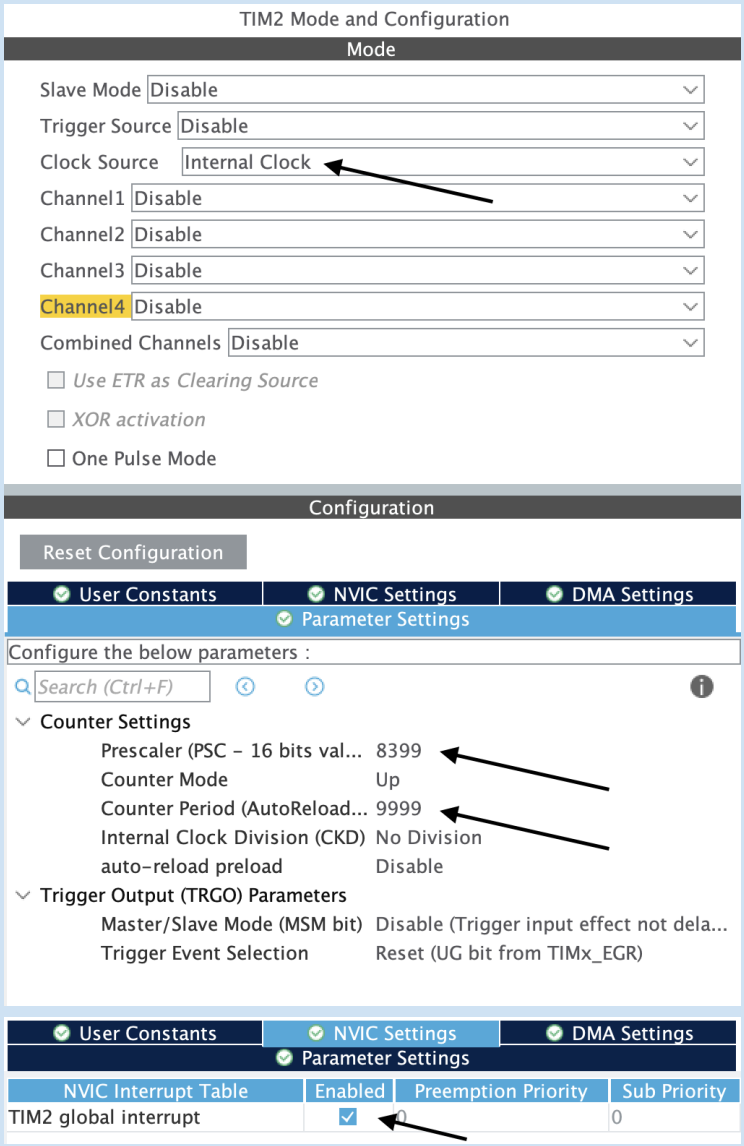
We then go into the connectivity section to enable the I2C1 and enable both the DMA to receive and the interrupts (required to be enabled as said in the documentation for the DMA).



In the connectivity section we also enable the USART2 and we enable the interrupt and the DMA in transmission (as done in previous homeworks).



Now to trigger the reading every second we have to configure TIM2 and its global interrupt. We set the clock source, the prescaler at 8399, the counter at 9999 and enable the global interrupt.



We now go into the “main.c” file. In the main function we just start the timer.

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim2);
/* USER CODE END 2 */
```

Then we just need to write the callback for the timer. We receive using the DMA function the two bytes of the temperature sensor. Convert them into a float value (the conversion we performed works for both positive and negative value since the cast to int16_t correctly interprets the two’s complement number correctly) and send it to the remote terminal as usual.

```c
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
  if (htim->Instance == TIM2) {
      HAL_I2C_Master_Receive_DMA(&hi2c1, LM75_ADDR, temp_data, 2);

      int16_t temp_raw = (int16_t)(temp_data[0] << 8 | temp_data[1]);

      float temperature = (temp_raw >> 5) * 0.125f;

      char string[64];
      int length = snprintf(string, sizeof(string), "Temp: %.3f C\n", temperature);
      HAL_UART_Transmit_DMA(&huart2, (uint8_t *)string, length);
  }
}
/* USER CODE END 0 */
```

To get it working we need to include the “stdio.h” library for the snprintf, enable the float in the printf, define “LM75_ADDR” and a variable “temp_data” as a destination in memory for the DMA.

```c
/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
#define LM75_ADDR (0x48 << 1)
/* USER CODE END PD */
```

```c
/* USER CODE BEGIN PV */
uint8_t temp_data[2];
/* USER CODE END PV */
```

We tested it and it seems to be working correctly.

Explain the "bug" of LM75B sensors and how you solved it.

The “bug” is caused by a race condition on the temperature registers.
In the B version of the LM75 the temperature reading is independent from the I2C communication, the sensor in normal mode performs temperature reading every 100ms and updates the registers.
If we just do the two byte read via I2C it could happen that we read the MSB byte from a previous reading, then the register updates, and we read the LSB byte from the new reading.

From reading the datasheet we found out that there is a shutdown mode, where the temperature reading stops and the last read value is kept in the registers.

Table 8. Conf register *...continued*
*Legend: * = default value.*

Bit	Symbol	Access	Value	Description
B2	OS_POL	R/W		OS polarity selection
			0*	OS active LOW
			1	OS active HIGH
B1	OS_COMP_INT	R/W		OS operation mode selection
			0*	OS comparator
			1	OS interrupt
B0	SHUTDOWN	R/W		device operation mode selection
			0*	normal
			1	shutdown

So to solve the “bug” we could send a write to modify the configuration register and set the shutdown bit to 1, perform the read and then send another write to set the shutdown bit back to 0 and restart normal operation. This bug does not happen in the non B version because the I2C communication interrupts the temperature reading.	
Professor comments:	